README FILE PART3

Student Number: ST10476705
Module: PROG6212 — Programming Assignment 3
Assignment: Part 3 — MVC Web Front-End, Authentication, and Containerisation

Overview

The RaceDay Web is an ASP.NET Core 8 MVC application that serves as the front-end for the RaceDay event management system. It consumes the RESTful API delivered in Part 2 and provides a complete, role-aware user interface for Organisers and Participants.

The web application does not access the database directly. All data operations — authentication, event management, category management, enrolments, and results — are performed through the Part 2 API using a typed `HttpClient` service. This keeps the web project strictly focused on presentation, session management, and user experience, while all business logic and persistence remain in the API.

The application supports two roles. An Organiser can manage events, define categories, view enrolments, and capture results. A Participant can browse events, enrol in categories, view their enrolments and race history, and update their profile. Users authenticate through the web application, which then stores their identity and role in a server-side session. All protected pages are guarded by a custom role-based authorisation filter.

Purpose

This application forms Part 3 of the PROG6212 programming assignment. Its purpose is to provide the user interface that consumes the Part 2 API, implementing session-based authentication, role-specific navigation, and all Organiser and Participant features required by the assignment brief.

The application has four main goals. First, to provide a complete MVC front-end that consumes the API for all data operations. Second, to implement session-based authentication with role-aware navigation. Third, to provide distinct, fully featured interfaces for Organisers and Participants. Fourth, to be containerised using Docker and validated automatically through GitHub Actions.

Technologies Used

The following technologies were used to build the web application.

ASP.NET Core MVC version 8.0 was used as the framework for the web application.

C# version 12 was used as the programming language.

Razor Views were used to render the user interface.

Session Middleware was used for session-based authentication.

A typed HttpClient was used to consume the Part 2 API.

Bootstrap 5 was used for responsive styling.

GitHub Actions was used for continuous integration.

Docker was used for containerisation.

Project Structure

The project is organised into the following folders.

The Controllers folder contains all MVC controllers, including AccountController for register, login, and logout, HomeController for landing and error pages, OrganiserController for all Organiser features, and ParticipantController for all Participant features.

The Filters folder contains RoleAuthorizeAttribute, the custom authorisation filter that enforces role-based access to controllers.

The Models folder contains the API DTO classes used for serialisation and deserialisation, including AuthResponse, EventDto, CategoryDto, EnrolmentDto, and ResultDto.

The ViewModels folder contains view-specific models such as OrganiserDashboardVm, ParticipantHomeVm, LoginVm, and RegisterVm.

The Services folder contains ApiService, the typed HttpClient service responsible for all communication with the Part 2 API, including session storage and retrieval.

The Views folder contains Razor views for each controller, organised into Account, Home, Organiser, and Participant subfolders.

The Program.cs file is the entry point of the application. It configures MVC, session state, the HTTP context accessor, the typed HttpClient, and the middleware pipeline.

The appsettings.json file contains the API base URL configuration.

The Dockerfile defines the multi-stage build for containerisation.

This separation of concerns keeps the code organised, maintainable, and easy to test.

Prerequisites

Visual Studio 2022 version 17.8 or later with the ASP.NET and web development workload installed.

The .NET 8.0 SDK, available from the Microsoft website.

The RaceDay API from Part 2 must be running and reachable. By default the API runs on http://localhost:5148.

Setup and Running

Step one is to clone the repository. Open a terminal and run the git clone command for the repository URL, then navigate into the project folder.

Step two is to restore the NuGet packages. Open the solution in Visual Studio, right click the solution in Solution Explorer, and choose Restore NuGet Packages. Alternatively, open the Package Manager Console and run dotnet restore.

Step three is to configure the API base URL. Open the appsettings.json file and confirm that ApiSettings BaseUrl points to the running Part 2 API. The default is http://localhost:5148.

Step four is to start the Part 2 API in a separate terminal or Visual Studio instance.

Step five is to run the web application. Open the RaceDay.Web project in Visual Studio and press F5 or Ctrl F5. The browser opens automatically at the configured localhost address.

The application does not create or manage any database. All persistence is handled by the API.

Authentication

Authentication is handled using server-side sessions and the JWT tokens issued by the Part 2 API.

When a user registers, the web application posts the registration details to POST api auth register. The API creates the user, hashes the password, and returns the user's details. The web application then automatically logs the user in by calling POST api auth login.

When a user logs in, the web application posts the credentials to POST api auth login. The API validates the credentials and returns a JWT token along with the user's identifier, full name, email, and role. The web application stores these values in the server-side session, including the role which is normalised to ensure consistent casing and spelling.

Every subsequent request to the web application reads the role from session to determine what the user is allowed to do and where they should be redirected.

The session expires after sixty minutes of inactivity by default. The session cookie is marked as HttpOnly and IsEssential.

Session and Role Normalisation

The role returned by the API is normalised before being stored in session. This ensures that values such as Organiser and Organizer, or inconsistent casing, are all mapped to the single canonical value Organiser. Any value that does not contain the substring organi is treated as Participant. This prevents a role typo or spelling difference from causing a logged-in user to be routed to the wrong dashboard.

Role Based Access Control

The web application enforces two roles through a custom authorisation filter called RoleAuthorizeAttribute.

An Organiser can access the Organiser dashboard, create, edit, and delete events, upload event banners, manage categories, view all enrolments for their events, and capture and publish results.

A Participant can access the home page and filter upcoming events, view full event details and available categories, enrol in a category, view their own enrolments, view their own race history, and update their profile.

The RoleAuthorizeAttribute is applied at the controller level. Any request to a protected controller without a valid session is redirected to the login page. Any request by a logged-in user to a controller they are not authorised for is redirected to their own dashboard.

Navigation links in the layout view are rendered conditionally based on the role stored in session, so each role only sees the links relevant to them.

Post-Login Redirect

After a successful login, the web application routes the user to the correct dashboard based on their normalised role. Organisers are redirected to the Organiser Dashboard. Participants are redirected to the Participant Home. This routing is handled by a single helper method to ensure consistency, and it prevents the login flow from sending an Organiser to the Participant page.

Organiser Interface

The Organiser dashboard shows a summary of all events the Organiser has created, the total number of enrolments per event, and the dates of upcoming events. Each event row includes quick-action links to view its enrolments or capture its results.

The Events management page allows the Organiser to create, edit, and delete events. Each event includes a name, description, date, location, distance, and event type. When creating or editing an event, the Organiser may upload a banner image, which is streamed to the API and stored in Azure Blob Storage. The resulting public URL is saved on the event record.

The Categories management page allows the Organiser to add age and distance categories to any of their events, and to remove categories that are no longer needed.

The Enrolments page allows the Organiser to view all participants who have enrolled for a specific event, including each participant's selected category and current enrolment status.

The Results entry page allows the Organiser to capture finish times and finishing positions for enrolled participants after an event concludes. Once saved, the result becomes immediately visible to the corresponding participant on their My Results page.

Participant Interface

The Participant home page shows all upcoming events with a filter option to narrow the list by keyword.

The Event details page shows the full information for an event, including its banner, description, date, location, distance, event type, and all available categories. Each category has an Enrol button.

Enrolling posts the event and selected category to the API and creates the enrolment immediately. The participant is then redirected to their My Enrolments page with a confirmation message.

The My Enrolments page shows a personal view of all events the participant has enrolled in, along with each enrolment's current status.

The My Results page shows the participant's personal race history across all completed events they were enrolled in. For each completed event, the page displays the event name, the event date, the category, the recorded finish time, and the finishing position formatted for example as 47th out of 312 finishers.

The Profile page allows the participant to view and update their personal information and upload a profile picture. Profile pictures follow the same Azure Blob Storage pipeline as event banners.

API Endpoints Consumed

The following endpoints from the Part 2 API are consumed by the web application.

Authentication. POST api auth register is used for registration. POST api auth login is used for login.

Events. GET api events lists events. GET api events by id returns an event with its categories. GET api events by organiser returns an organiser's own events. POST api events creates an event. PUT api events by id updates an event. DELETE api events by id deletes an event.

Categories. GET api events by event id categories lists categories. POST api events by event id categories adds a category. PUT api events by event id categories by id updates a category. DELETE api events by event id categories by id removes a category.

Enrolments. GET api enrolments by event returns all enrolments for an event. GET api enrolments by participant returns a participant's own enrolments. POST api enrolments

creates an enrolment. PUT api enrolments by id updates an enrolment. DELETE api enrolments by id cancels an enrolment.

Results. GET api results by event returns all results for an event. POST api results captures a result. PUT api results by id updates a result. DELETE api results by id deletes a result.

Users. GET api users by id returns a user's profile. PUT api users by id updates a user's profile. DELETE api users by id deletes a user's account.

Blob. POST api blob upload uploads an image to Azure Blob Storage and returns its public URL.

Every protected endpoint requires a Bearer token in the Authorization header. The web application transparently attaches the token stored in session.

Azure Blob Storage Integration

Image uploads are handled by the Part 2 API. When the Organiser uploads an event banner or the Participant uploads a profile picture, the web application streams the file to the API as multipart form data. The API uploads the file to the raceday-images container in Azure Blob Storage and returns the public URL. The web application stores the URL against the relevant entity. This design keeps blob storage credentials entirely within the API and respects the principle of least privilege.

Docker

The web application is fully containerised using a multi-stage Dockerfile.

The build stage uses the dotnet sdk 8.0 image to restore packages, compile the project, and publish the output.

The runtime stage uses the dotnet aspnet 8.0 image to host the published output on port 8080.

The application checks the DOTNET_RUNNING_IN_CONTAINER environment variable and skips HTTPS redirection when running inside a container, so HTTP-only container traffic works correctly without redirect loops.

To build the image, run the docker build command with the tag raceday web and the path to the RaceDay.Web project.

To run the image, use the docker run command mapping port five thousand on the host to port eight thousand eight hundred and eighty inside the container, and set the ApiSettings BaseUrl environment variable to point to the running Part 2 API.

Continuous Integration

The project uses GitHub Actions for continuous integration. The workflow is split into three parts.

Part one validates the documentation structure, including the presence of the docs folder, the ERD, the API endpoint plan, the SQL script, and the README file.

Part two validates the Part 2 API project, including controllers, models, JWT authentication wiring, SQL schema, and a successful build.

Part three validates the Part 3 MVC web project, including the presence of the session middleware, the typed HttpClient registration, the ApiSettings configuration, the ApiService implementation, the RoleAuthorizeAttribute, the Organiser and Participant controllers, the role-based redirect logic, the required Razor views, the Dockerfile, and a successful build.

On every push or pull request to the main branch, GitHub checks out the repository, installs the .NET 8 SDK, restores NuGet packages, runs the validation steps, and builds both the API and the web projects.

If any step fails, the commit shows a red cross. If all steps succeed, a green check appears on the commit.

The workflow status can be seen on the repository's Actions tab.

Testing the Application Manually

The following sequence demonstrates the main features of the application.

First, start the Part 2 API.

Second, start the web application.

Third, open the web application in the browser. The home page is displayed.

Fourth, register a new user as an Organiser. The application logs the user in and redirects to the Organiser Dashboard.

Fifth, create an event from the Events page, uploading a banner image. The event appears on the dashboard with an enrolment count of zero.

Sixth, add a category to the event from the Categories page.

Seventh, log out. The session is cleared and the user is redirected to the home page.

Eighth, register a new user as a Participant. The application logs the user in and redirects to the Participant Home.

Ninth, browse the event list, filter by keyword, and open the event details page.

Tenth, enrol in a category. The enrolment is created and the participant is redirected to My Enrolments with a confirmation message.

Eleventh, log out and log back in as the Organiser. Open the Enrolments page for the event and confirm that the participant appears with the correct category and status.

Twelfth, open the Results page for the event, capture a finish time and finishing position for the participant, and save the result.

Thirteenth, log out and log back in as the Participant. Open My Results and confirm that the event appears with the correct finish time and position.

Error Handling

The web application handles errors gracefully.

If a user attempts to access a protected page without being logged in, they are redirected to the login page.

If a user attempts to access a page for a role they do not have, they are redirected to their own dashboard.

If the API returns an error, the web application displays an appropriate message to the user and logs the failure.

If an unhandled exception occurs, the application redirects to the Home Error page with an appropriate status code.

Security Considerations

Session cookies are marked HttpOnly and IsEssential to mitigate cross-site scripting and to ensure functionality.

All state-changing operations such as login, register, logout, create, edit, delete, enrol, and save result are protected with anti-forgery tokens.

The web application never holds database credentials, SQL connection strings, or Azure Blob Storage credentials. These are exclusively held by the API.

The API base URL is read from configuration and can be overridden by an environment variable, allowing credentials and endpoints to be supplied safely in production environments.

Role strings are normalised before being used for authorisation decisions, preventing role-mismatch attacks caused by case or spelling differences.

Known Limitations

There is no refresh token mechanism, so users must log in again after their API token expires.

There is no pagination on list pages.

There is no real-time notification when results are published; participants must refresh the page.

The API base URL is configured per environment rather than being resolved dynamically.

Future Improvements

If the project is extended, the following features would be added.

A refresh token endpoint and flow for smoother sessions.

Email verification on registration.

Pagination on list pages.

A participant notification system for event changes and result publication.

Structured logging using Serilog.

API versioning using the api v1 path.

Automated end-to-end tests using Playwright or Selenium.

References

Microsoft Corporation (2023) ASP.NET Core MVC Documentation. Available at https://learn.microsoft.com/en-us/aspnet/core/mvc/ accessed on 20 August 2026.

Microsoft Corporation (2023) Session and State Management in ASP.NET Core. Available at https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state accessed on 20 August 2026.

Microsoft Corporation (2023) Docker Images for ASP.NET Core. Available at https://learn.microsoft.com/en-us/dotnet/core/docker/ accessed on 20 August 2026.

Freeman, A. (2022) Pro ASP.NET Core 7. Tenth edition. New York: Apress.

Fowler, M. (2010) Patterns of Enterprise Application Architecture. Boston: Addison Wesley.

Richardson, L. and Amundsen, M. (2013) RESTful Web APIs: Services for a Changing World. Sebastopol: O'Reilly Media.